$\vec{\nabla} f$ Gradient:

It tells us how fast a scalar changes in different directions.
The operator takes a scalar field and returns a vector field.

1D $\frac{df}{dx}$

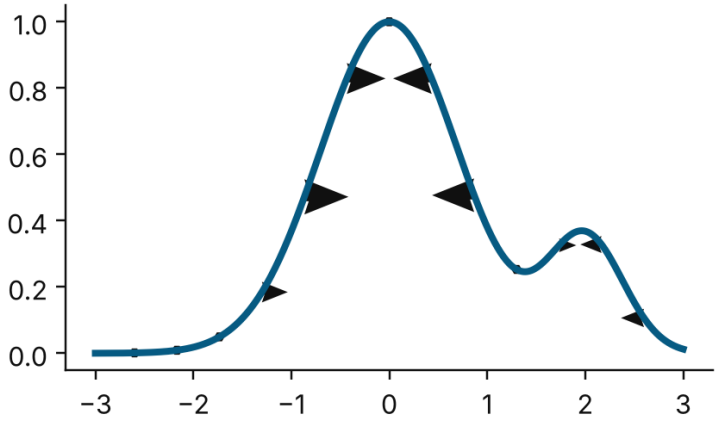
2D $\vec{\nabla} f = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right)$

3D $\vec{\nabla} f = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z} \right)$

Magnitude $|\vec{\nabla} f| = \sqrt{\left(\frac{\partial f}{\partial x} \right)^2 + \left(\frac{\partial f}{\partial y} \right)^2}$

- Direction: the steepest increase, straight uphill.
- Always perpendicular to the contours of f.

In 1D it is simply the slope of the curve.



Arrows point towards increasing f.

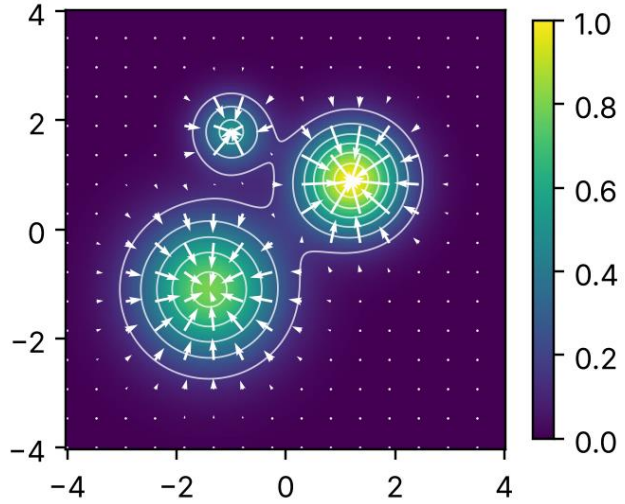
```
gradient_rho = np.gradient(rho_cgs2D, dx[0])

xgradient_rho = gradient_rho[0]
ygradient_rho = gradient_rho[1]

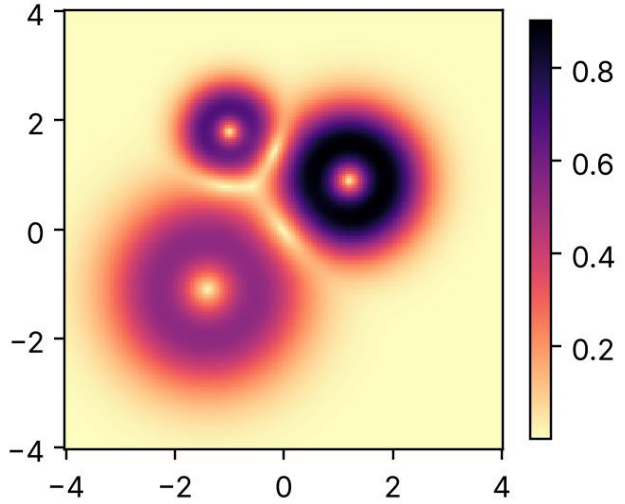
modulus_gradient_rho = np.sqrt(
    xgradient_rho**2 + ygradient_rho**2)
```

$\vec{\nabla} f$ Gradient in 2D

$f(x, y)$ with its gradient arrows



$|\vec{\nabla} f|$ is large only at the edges



- The arrows run from low to high f and cross the contours at right angles.
- The magnitude is large only where f changes fast, so it works as an edge detector.

```
fig, axs = plt.subplots(1,2)

axs[0].pcolor(X, Y, rho_cgs2D,
              cmap='viridis')

axs[1].quiver(X, Y, xgradient_rho,
              ygradient_rho,
              modulus_gradient_rho,
              cmap='Oranges')
```

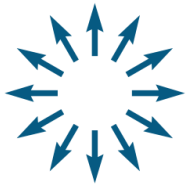
$\vec{\nabla} \cdot \vec{v}$ Divergence:

It tells us how much a vector field spreads out or converges at a point.
The operator takes a vector field and returns a scalar field.

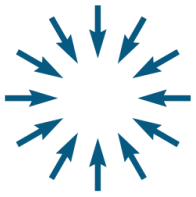
1D $\frac{dv}{dx}$

2D $$\vec{\nabla} \cdot \vec{v} = \begin{pmatrix} \partial/\partial x \\ \partial/\partial y \end{pmatrix} \cdot \begin{pmatrix} v_x \\ v_y \end{pmatrix} = \frac{\partial v_x}{\partial x} + \frac{\partial v_y}{\partial y}$$

3D $$\vec{\nabla} \cdot \vec{v} = \begin{pmatrix} \partial/\partial x \\ \partial/\partial y \\ \partial/\partial z \end{pmatrix} \cdot \begin{pmatrix} v_x \\ v_y \\ v_z \end{pmatrix} = \frac{\partial v_x}{\partial x} + \frac{\partial v_y}{\partial y} + \frac{\partial v_z}{\partial z}$$



source



sink

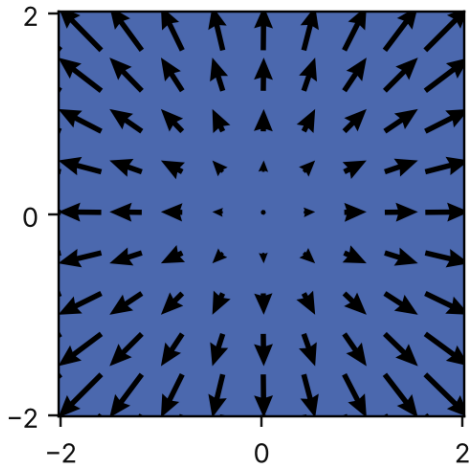
```
def get_divergence(comp1, comp2, dx):
    div1 = np.gradient(comp1, dx[0], axis=1)
    div2 = np.gradient(comp2, dx[0], axis=0)
    divt = div1 + div2
    return divt
```

- The net outflow per unit volume at a point.
- Positive is a source, negative is a sink, zero means nothing is created or lost.

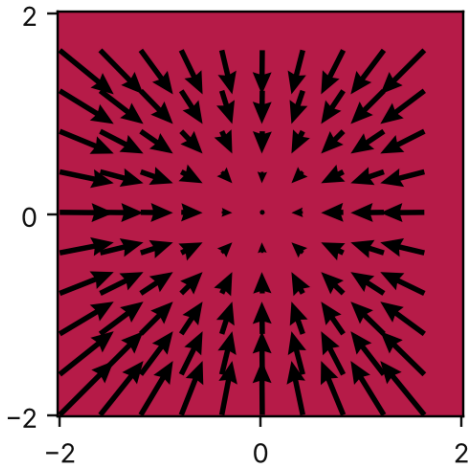
Check on the simulation data: $\vec{\nabla} \cdot \vec{B} = 0$

$\vec{\nabla} \cdot \vec{v}$ Divergence in 2D

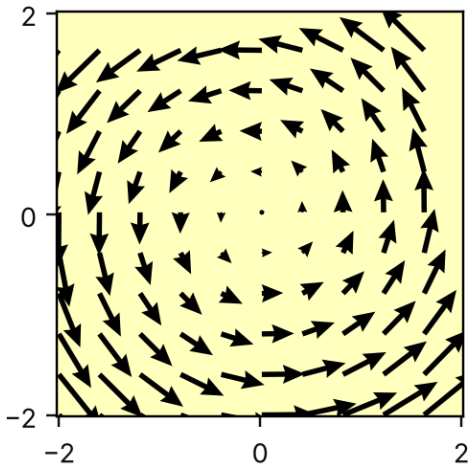
outflow $\vec{v} = (x, y)$



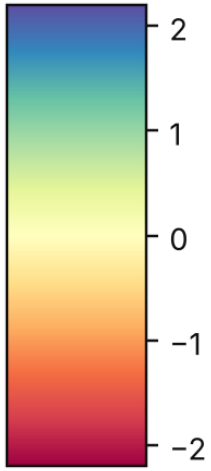
inflow $\vec{v} = (-x, -y)$



rotation $\vec{v} = (-y, x)$



$\vec{\nabla} \cdot \vec{v}$



- Blue is positive, the field spreads out; red is negative, the field converges.
- Pure rotation has zero divergence: it moves the gas around but creates none.

```
div_bfield = get_divergence(Bx1_cgs2D,
                            Bx2_cgs2D, dx)

plt.pcolor(X, Y, div_bfield,
           cmap='Spectral', shading='auto')
plt.colorbar()
plt.show()
```

$\vec{\nabla} \times \vec{v}$ Curl:

It tells us how much a vector field rotates around a point.
The operator takes a vector field and returns another vector field.

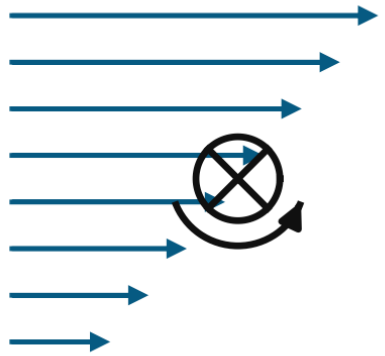
1D no curl — rotation needs at least two directions

2D $$(\vec{\nabla} \times \vec{v})_z = \begin{vmatrix} \partial/\partial x & \partial/\partial y \\ v_x & v_y \end{vmatrix} = \frac{\partial v_y}{\partial x} - \frac{\partial v_x}{\partial y}$$

3D $$\vec{\nabla} \times \vec{v} = \begin{vmatrix} \hat{\imath} & \hat{\jmath} & \hat{k} \\ \partial/\partial x & \partial/\partial y & \partial/\partial z \\ v_x & v_y & v_z \end{vmatrix} = \begin{pmatrix} \partial v_z/\partial y - \partial v_y/\partial z \\ \partial v_x/\partial z - \partial v_z/\partial x \\ \partial v_y/\partial x - \partial v_x/\partial y \end{pmatrix}$$

- Local spin: drop a tiny paddle wheel into the flow and see if it turns.
- Zero curl everywhere means the flow is irrotational.

In fluids this is the vorticity: $\vec{\omega} = \vec{\nabla} \times \vec{v}$

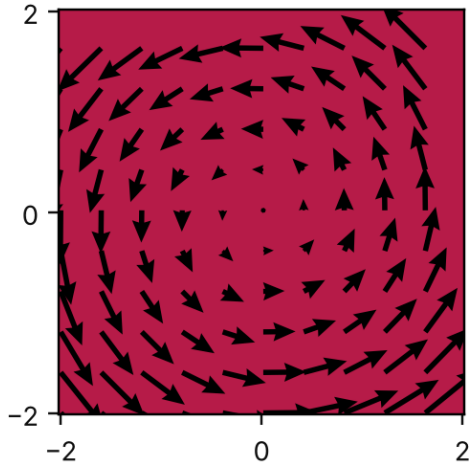


A shear flow spins the paddle wheel.

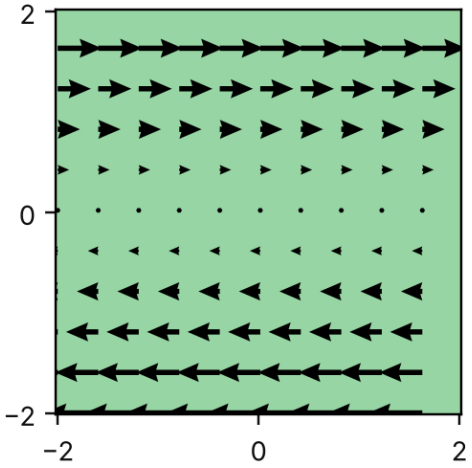
```
def get_curl(comp1, comp2, dx):
    curl1 = np.gradient(comp1, dx, axis=0)
    curl2 = np.gradient(comp2, dx, axis=1)
    curlt = curl2 - curl1
    return curlt
```

$\vec{\nabla} \times \vec{v}$ Curl in 2D

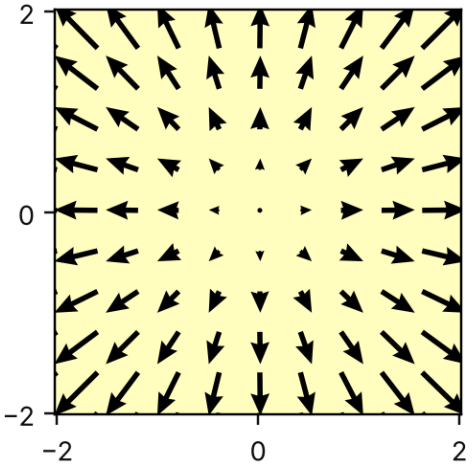
rotation $\vec{v} = (-y, x)$



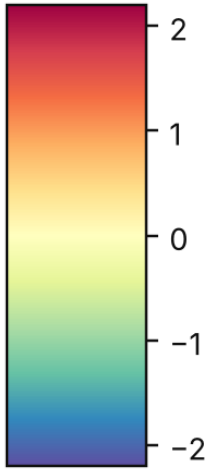
shear $\vec{v} = (y, 0)$



radial $\vec{v} = (x, y)$



$(\vec{\nabla} \times \vec{v})_z$



- Rotation spins the wheel one way, shear spins it the other, even though its arrows are straight.
- A purely radial field has zero curl, the mirror image of the divergence slide.

```
curl_vel = get_curl(vx1_cgs2D, vx2_cgs2D,
                    dx[0])

plt.pcolor(X, Y, curl_vel,
           cmap='Spectral_r',
shading='auto')
plt.colorbar()
plt.show()
```